

Consensus Knowledge Refinery: A Technical Proposal for Multi-Agent Knowledge Compression Before Foundation Model Training

Research Proposal Draft

Version 1.0

June 2026

Abstract

Large language models are commonly trained on corpora containing repeated facts, paraphrases, stale answers, contradictions, and long-tail noise. This proposal studies a pretraining pipeline called *Consensus Knowledge Refinery* (CKR): a multi-agent process that extracts claims, clusters semantically equivalent claims, estimates source quality and evidence, preserves rare but important facts, and resolves or flags contradictions before a student model is trained. The paper is a research proposal rather than a completed empirical result. Its central hypothesis is that models can improve knowledge density and reduce wasted training tokens if the corpus is refined at the concept level, while preserving provenance and uncertainty. We formalize signed agreement, entropy-based uncertainty, source quality, evidence strength, rarity, and conflict penalties; define auditing metrics for accepted and rejected cards; and propose ablations that separate data-refinement effects from architecture effects such as sparse attention. The proposal is deliberately conservative: it does not claim that a small model will generally match frontier-scale systems, but it outlines experiments to test whether a smaller model can behave like a stronger domain-specialist model under high-quality compressed training data.

Status. This is a technical proposal. The formulas and algorithms are intended as a starting specification for experiments, not as a proven training recipe.

1 Motivation

The next wave of efficient language modeling may not come only from larger parameter counts. It may also come from better handling of training data. A web-scale corpus often contains the same answer many times, low-quality paraphrases of high-quality sources, old answers that no longer work, and conflicting claims presented without explicit provenance. Direct training can spend capacity repeatedly fitting duplicated or weak evidence.

CKR asks a narrower question: can we train on *refined knowledge units* rather than raw text? The refinement process is not a single model deciding truth. It is an ensemble-based process: multiple specialist agents extract, vote, critique, and attach provenance. The output is a data card that can be accepted, rejected, or preserved as disputed.

This direction is related to data curation and distillation, but the object being compressed is different. Knowledge distillation compresses a teacher model into a student model [10]. Dataset

distillation compresses training examples into smaller synthetic sets [11]. CKR compresses and organizes *knowledge before training*; it is closer to a pretraining data refinery than to model compression.

2 Related Work

Transformers established attention-based sequence modeling as the dominant language-model backbone [1]. Efficient attention work such as FlashAttention reduces memory movement and improves practical training throughput while preserving exact dense attention [2]. SubQ-1.1-Small is included here as a vendor technical report on Subquadratic Sparse Attention (SSA), claiming linear compute and memory in sequence length; because it is not peer-reviewed, this proposal treats it as a relevant but externally unvalidated technical source [3]. Recurrent and state-space alternatives, including Mamba and RWKV, target long-context efficiency through stateful or recurrent structure [4, 5]. Low-bit approaches such as BitNet and BitNet b1.58 explore native low-precision training and inference [6, 7, 8], while inference stacks such as bitnet.cpp focus on practical deployment efficiency [9]. Sutton’s “Bitter Lesson” is included as a caution: hand-designed intelligence pipelines often lose to scalable computation, so CKR must be validated empirically rather than assumed superior [12].

3 Overview

Let $\mathcal{D}$ denote the raw corpus and $\mathcal{R}$ the refined corpus produced by CKR. Each document in $\mathcal{D}$ is routed to one or more specialist agents. Agents produce claim-level judgments, not final training examples. Claims are clustered into semantic groups $g \in \mathcal{G}$. Each cluster is scored using signed agreement, source quality, evidence strength, rarity, conflict, and uncertainty. A data-card generator then emits accepted, rejected, uncertain, or disputed examples.

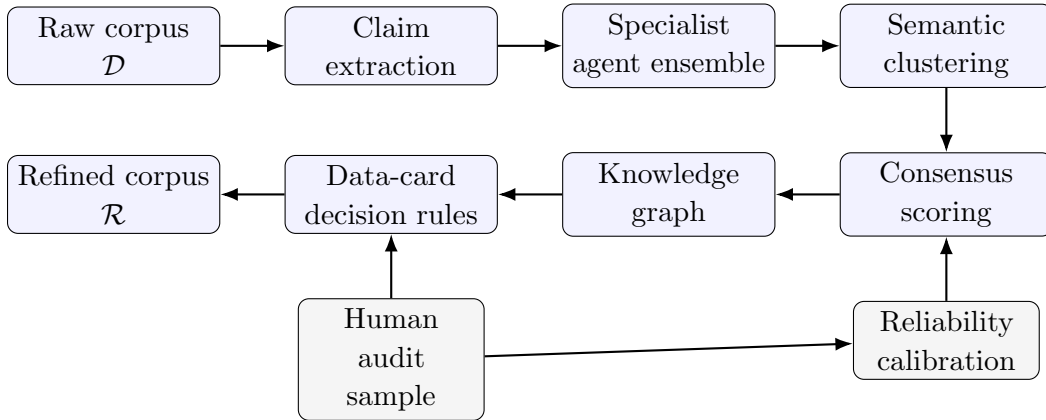


Figure 1: CKR pipeline. The system refines a raw corpus $\mathcal{D}$ into a provenance-preserving refined corpus $\mathcal{R}$.

4 Notation and Symbol Discipline

The proposal intentionally separates symbols that earlier drafts could overload. Raw corpus is $\mathcal{D}$; refined corpus is $\mathcal{R}$; rarity is $\text{Rar}(g)$, not $R(g)$; source quality is $\text{Src}(g)$, not $S(g)$; evaluation

score is $\text{Eval}(M, q)$; evidence strength is $\text{Ev}(g)$; expectation is $\mathbb{E}[\cdot]$; the vote label is $\nu \in \{+, -, 0\}$; the agent index is i ; sparse attention fan-out is κ .

Symbol	Meaning
$\mathcal{D}$	Raw input corpus
$\mathcal{R}$	Refined corpus produced by CKR
$g \in \mathcal{G}$	Semantic claim cluster
$i \in \{1, \dots, m\}$	Specialist agent index
d	Domain label such as coding, math, legal, science
$\Theta_{i,d}$	Reliability of agent i in domain d
q_i, e_i, r_i	Agent confidence, evidence support, and relevance
$\nu_i \in \{+, -, 0\}$	Agent vote: support, reject, or abstain/uncertain
$A(g)$	Signed agreement in $[-1, 1]$
$\text{Unc}_{\text{norm}}(g)$	Entropy-normalized uncertainty in $[0, 1]$
$\text{Src}(g)$	Source-quality score in $[0, 1]$
$\text{Ev}(g)$	Evidence strength in $[0, 1]$
$\text{Rar}(g)$	Rarity-preservation score in $[0, 1]$
$C_t(g)$	Conflict score at fixed-point iteration t
$\mathcal{K}_t(g)$	Confidence score of cluster g at iteration t
$\text{Eval}(M, q)$	Score of model M on evaluation item q

Table 1: Symbol table used throughout this proposal.

5 Agent Judgments

Each agent returns a judgment tuple

$$J_i(c, d) = (\nu_i, q_i, e_i, r_i, \ell_i), \quad (1)$$

where $\nu_i \in \{+, -, 0\}$ is the signed vote, $q_i \in [0, 1]$ is confidence, $e_i \in [0, 1]$ is evidence support, $r_i \in [0, 1]$ is domain relevance, and ℓ_i is a natural-language rationale.

The domain-specific reliability $\Theta_{i,d}$ is learned from audit outcomes rather than fixed globally. A coding agent may be reliable in Python but unreliable in legal interpretation. This prevents a single global reliability score from over-generalizing.

6 Consensus Components Without Evidence Double Counting

Vote masses exclude evidence e_i so that evidence can remain a separate term. This avoids double counting evidence through agreement and through the evidence score.

$$M^+(g) = \sum_i \Theta_{i,d} q_i r_i \mathbf{1}[\nu_i = +], \quad (2)$$

$$M^-(g) = \sum_i \Theta_{i,d} q_i r_i \mathbf{1}[\nu_i = -], \quad (3)$$

$$M^0(g) = \sum_i \Theta_{i,d} q_i r_i \mathbf{1}[\nu_i = 0]. \quad (4)$$

Signed agreement preserves vote direction:

$$A(g) = \frac{M^+(g) - M^-(g)}{M^+(g) + M^-(g) + \epsilon}. \quad (5)$$

A cluster rejected by all agents receives a negative agreement score, not a high positive score.

Uncertainty is entropy-based rather than $1 - A(g)$:

$$\text{Unc}(g) = - \sum_{\nu \in \{+, -, 0\}} p_\nu(g) \log(p_\nu(g) + \epsilon), \quad (6)$$

where $p_\nu(g) = M^\nu(g)/(M^+ + M^- + M^0 + \epsilon)$. The normalized form is

$$\text{Unc}_{\text{norm}}(g) = \frac{\text{Unc}(g)}{\log 3}. \quad (7)$$

Evidence strength is separate:

$$\text{Ev}(g) = \frac{\sum_i \Theta_{i,d} q_i e_i r_i}{\sum_i \Theta_{i,d} q_i r_i + \epsilon}. \quad (8)$$

Source quality is computed from provenance and trust scores:

$$\text{Src}(g) = \frac{\sum_{s \in \mathcal{S}_g} w_s Q_s}{\sum_{s \in \mathcal{S}_g} w_s + \epsilon}. \quad (9)$$

Here $Q_s \in [0, 1]$ is the source-quality estimate and w_s is the weight assigned to source s based on recency, authority, and provenance completeness.

Rarity is normalized to avoid dominating the sigmoid input:

$$\text{Rar}(g) = \text{clip} \left(\frac{I(g)}{1 + \log(1 + f(g))}, 0, 1 \right), \quad I(g) \in [0, 1]. \quad (10)$$

where $I(g)$ is an importance score and $f(g)$ is frequency. Rare facts are preserved only when they have enough importance and evidence.

7 Conflict Score and Fixed-Point Estimation

Let $\mathcal{N}^-(g)$ be clusters that contradict g , and let $\text{neg}(g, h) \in [0, 1]$ be a contradiction-strength function. A fully normalized mean removes degree bias but may under-penalize claims contradicted by many independent clusters. This proposal uses sub-linear breadth-sensitive normalization:

$$C_t(g) = 1 - \exp \left(- \frac{\sum_{h \in \mathcal{N}^-(g)} \mathcal{K}_t(h) \text{neg}(g, h)}{\sqrt{|\mathcal{N}^-(g)|} + \epsilon} \right). \quad (11)$$

This keeps $C_t(g) \in [0, 1]$ while allowing broad contradiction to matter more than a single weak contradiction.

The cluster confidence is computed iteratively:

$$\hat{\mathcal{K}}_{t+1}(g) = \sigma\left(\alpha A(g) + \beta \text{Src}(g) + \gamma \text{Ev}(g) + \delta \text{Rar}(g) - \lambda C_t(g) - \eta \text{Unc}_{\text{norm}}(g)\right). \quad (12)$$

The fixed point is not proven unique. The implementation should use damping, convergence thresholds, and maximum-iteration caps:

$$\mathcal{K}_{t+1} \leftarrow \rho \mathcal{K}_t + (1 - \rho) \hat{\mathcal{K}}_{t+1}, \quad \rho \in [0, 1]. \quad (13)$$

8 Decision Rules

Data cards are emitted after scoring. A strong contradiction is defined explicitly as

$$\text{StrongConflict}(g) \iff C_t(g) \geq \theta_{\text{dispute}} \quad \text{or} \quad \max_{h \in \mathcal{N}^-(g)} \text{neg}(g, h) \geq \tau_{\text{neg}}. \quad (14)$$

The threshold τ_{neg} is the minimum pairwise contradiction strength used to create a negative edge in the knowledge graph. The clustering threshold τ_{sim} is used in semantic clustering: two claims are placed in the same cluster when their semantic similarity exceeds τ_{sim} and they do not cross the contradiction threshold τ_{neg} .

Algorithm 1 Data-card decision rules

Require: Cluster g , score $\mathcal{K}(g)$, conflict $C(g)$, thresholds $\theta_{\text{accept}}, \theta_{\text{reject}}, \theta_{\text{dispute}}$

- 1: **if** $\text{StrongConflict}(g)$ **then**
 - 2: emit $\text{DisputedClaimCard}(g)$ with both supporting and opposing provenance
 - 3: **else if** $\mathcal{K}(g) \geq \theta_{\text{accept}}$ **then**
 - 4: emit $\text{AcceptedCard}(g)$
 - 5: **else if** $\mathcal{K}(g) \leq \theta_{\text{reject}}$ **then**
 - 6: emit $\text{RejectedCard}(g)$
 - 7: **else**
 - 8: emit $\text{UncertainCard}(g)$ for audit or deferred processing
 - 9: **end if**
-

9 Data-Card Schema

The output card must include enough provenance to support later audit. The schema includes the execution safety class for code examples.

Field	Required	Meaning
topic	yes	Domain or task cluster
claim	yes	Canonical claim or solution statement
answer	yes	Final training answer or explanation
confidence	yes	$\mathcal{K}(g)$ after fixed-point scoring
rarity	yes	$\text{Rar}(g)$ rarity-preservation score
status	yes	accepted, rejected, disputed, or uncertain
safety	yes	preview-only, sandbox-only, safe-to-run, or unsafe
provenance.sources	yes	Source URLs, document IDs, or dataset IDs
provenance.license	yes	License or usage status
provenance.collection_date	yes	Date collected
provenance.agent_votes	yes	Vote summary and rationales
provenance.contradiction_links	yes	Negative-edge references
provenance.audit_status	yes	unaudited, sampled, passed, failed

Table 2: Minimum data-card schema.

Listing 1: Example data card aligned with the schema.

```
{
  "topic": "Python order-preserving deduplication",
  "claim": "dict.fromkeys preserves insertion order",
  "answer": "Use list(dict.fromkeys(x)) for ordered dedup.",
  "confidence": 0.91,
  "rarity": 0.38,
  "status": "accepted",
  "safety": "safe-to-run",
  "provenance": {
    "sources": ["python-docs", "stackoverflow-thread-..."],
    "license": "compatible",
    "collection_date": "2026-06-19",
    "agent_votes": {"support": 4, "reject": 0, "abstain": 1},
    "contradiction_links": [],
    "audit_status": "sampled"
  }
}
```

10 Metrics

Accuracy change is the lead metric:

$$\Delta_{\text{acc}} = \text{Acc}(M_{\text{ref}}) - \text{Acc}(M_{\text{raw}}). \quad (15)$$

A secondary relative metric is

$$KR_{\text{gt}} = \frac{\text{Acc}(M_{\text{ref}})}{\text{Acc}(M_{\text{raw}}) + \epsilon}. \quad (16)$$

The term KR_{gt} is a relative score, not literal retention percentage, and may exceed 1.

For rejected cards, two metrics are required to avoid ambiguity. First, rejected-correct fraction (RCF) asks: of rejected cards, how many were actually correct?

$$\text{RCF} = \frac{|\{b \in \mathcal{B}_{\text{reject}} : b \text{ is correct}\}|}{|\mathcal{B}_{\text{reject}}| + \epsilon}. \quad (17)$$

Second, correct-claim loss rate (CCLR) measures actual knowledge loss relative to all correct claims:

$$\text{CCLR} = \frac{|\{b \in \mathcal{B}_{\text{reject}} : b \text{ is correct}\}|}{|\{b \in \mathcal{B}_{\text{all}} : b \text{ is correct}\}| + \epsilon}. \quad (18)$$

RCF is a triage warning; CCLR is the knowledge-loss metric.

11 Training Cost Model

Let P be parameter count, T token count, and ξ hardware-training efficiency where higher ξ means lower cost. A simple planning approximation is

$$F_{\text{train}}(P, T) \approx \frac{cPT}{\xi}, \quad (19)$$

where c absorbs optimizer, sequence, and implementation constants. For the raw student:

$$F_{\text{student,raw}} \equiv F_{\text{train}}(P, T_{\mathcal{D}}). \quad (20)$$

If $\tau_{\text{data}} = |\mathcal{R}|/|\mathcal{D}|$ denotes refined-token fraction, then

$$F_{\text{student,ref}} \approx \tau_{\text{data}} F_{\text{student,raw}}. \quad (21)$$

Net savings must include the refinery cost:

$$\text{NetSavings} = 1 - \frac{F_{\text{refinery}} + F_{\text{student,ref}}}{F_{\text{student,raw}}} \approx 1 - \tau_{\text{data}} - \frac{F_{\text{refinery}}}{F_{\text{student,raw}}}. \quad (22)$$

This equation is useful only when F_{refinery} can be amortized across experiments or training runs.

12 Efficient Attention Integration

CKR reduces corpus-level training tokens; sparse attention reduces per-sequence attention cost. These are different axes. Let N_{seq} be the number of training sequences and L be average sequence length. Dense attention has approximate per-layer attention cost

$$O(N_{\text{seq}} L^2 d), \quad (23)$$

while sparse attention with κ selected links has approximate cost

$$O(N_{\text{seq}} L \kappa d). \quad (24)$$

If CKR reduces the number of training tokens by changing either sequence count or sequence length, the combined cost must be computed after measuring the actual refined sequence distribution. The proposal therefore treats

$$O(\tau_N N_{\text{seq}} L \kappa d) \quad (25)$$

as a planning model only when CKR primarily reduces sequence count by τ_N . If CKR shortens sequences instead, L must be re-estimated.

13 Experimental Design

The benchmark must separate data effects, model-size effects, and attention effects.

Arm	Size	Data	Attention	Purpose	Interpretation
A0	50M	raw	dense	cheap smoke test	catches pipeline failures
A1	50M	CKR	dense	tiny data effect	validates direction before larger runs
A2	125M	raw	dense	baseline	raw small model
A3	125M	CKR	dense	data effect	refined vs raw at same size
A4	125M	CKR-no-conflict	dense	conflict ablation	tests conflict penalty
A5	125M	CKR-no-rarity	dense	rarity ablation	tests rare-fact preservation
A6	500M	raw	dense	larger raw control	isolates size effect
A7	500M	CKR	dense	same-size data effect	A6 vs A7 isolates data at 500M
A8	500M	CKR	SSA/sparse	attention effect	A7 vs A8 isolates attention
A9	1B	CKR	dense	scaling pilot	tests mid-scale behavior
A10	4B	CKR	dense/sparse	optional final run	only after smaller validation

Table 3: Benchmark arms. The 4B target is deliberately staged after 50M, 125M, 500M, and 1B validation.

A key limitation is scale extrapolation: a 500M result does not prove a 4B result. The 4B run should be treated as a later-stage validation only after smaller arms show robust gains.

14 Statistical Testing

For benchmark questions $q_1, \dots, q_n$, paired comparisons should be reported:

$$\Delta_{\text{acc}} = \frac{1}{n} \sum_{j=1}^n [\mathbf{1}(M_{\text{ref}}(q_j) = z_j) - \mathbf{1}(M_{\text{raw}}(q_j) = z_j)], \quad (26)$$

where z_j is ground truth. Use bootstrap confidence intervals over items and report effect sizes separately for factuality, reasoning, and code execution.

15 Reliability Calibration

To avoid index collisions, this section keeps i for the agent and uses j for the calibration item. For calibration item j and agent i , let $p_{i,j} = q_{i,j}$ be the initial probability implied by that agent’s confidence. If agents output separate calibrated probabilities, those should replace $q_{i,j}$. With target $z_j \in \{0, 1\}$, the domain-specific Brier score is

$$B_{i,d} = \frac{1}{n_d} \sum_{j=1}^{n_d} (p_{i,j} - z_j)^2. \quad (27)$$

Reliability is updated with step size $\zeta \in (0, 1)$ and domain baseline b_d :

$$\Theta_{i,d} \leftarrow (1 - \zeta)\Theta_{i,d} + \zeta \left(1 - \frac{B_{i,d}}{b_d + \epsilon} \right)_+. \quad (28)$$

16 Formal Algorithms

Algorithm 2 Consensus Knowledge Refinery

Require: Corpus $\mathcal{D}$, thresholds $\tau_{\text{sim}}, \tau_{\text{neg}}$, agent ensemble $\mathcal{A}$

- 1: Initialize graph $\mathcal{G} \leftarrow \emptyset$
 - 2: **for** document $d \in \mathcal{D}$ **do**
 - 3: Extract candidate claims $c_1, \dots, c_m$
 - 4: **for** claim c **do**
 - 5: Route c to domain agents $\mathcal{A}_d$
 - 6: Collect judgments $J_i(c, d)$
 - 7: Assign c to cluster g using τ_{sim}
 - 8: Add negative edges when contradiction strength exceeds τ_{neg}
 - 9: Update source, evidence, and provenance records
 - 10: **end for**
 - 11: **end for**
 - 12: Run fixed-point scoring over clusters $g \in \mathcal{G}$ using Algorithm 3
 - 13: Emit data cards using Algorithm 1
 - 14: **return** refined corpus $\mathcal{R}$
-

Algorithm 3 Fixed-point scoring

Require: Clusters $\mathcal{G}$, maximum iterations T_{max} , tolerance τ , damping ρ

- 1: Initialize $\mathcal{K}_0(g) = 0.5$ for all $g \in \mathcal{G}$
 - 2: **for** $t = 0$ to $T_{\text{max}} - 1$ **do**
 - 3: **for** cluster $g \in \mathcal{G}$ **do**
 - 4: Compute $C_t(g)$ from contradiction neighbors
 - 5: Compute raw score $\hat{\mathcal{K}}_{t+1}(g)$ from the six-term equation
 - 6: Set $\mathcal{K}_{t+1}(g) = \rho\mathcal{K}_t(g) + (1 - \rho)\hat{\mathcal{K}}_{t+1}(g)$
 - 7: **end for**
 - 8: **if** $\max_{g \in \mathcal{G}} |\mathcal{K}_{t+1}(g) - \mathcal{K}_t(g)| < \tau$ **then**
 - 9: break
 - 10: **end if**
 - 11: **end for**
 - 12: **return** $\mathcal{K}(g)$
-

17 Execution Safety for Code Data

A coding dataset can teach unsafe commands if actionability is not separated from explanation. CKR uses the safety field: *preview-only* for commands that should not be run automatically, *sandbox-only* for commands requiring isolation, *safe-to-run* for harmless examples, and *unsafe* for rejected operational instructions. For example, Python’s `set(x)` does not preserve order for deduplication; `list(dict.fromkeys(x))` is preferred when insertion order matters.

18 Failure Modes

Failure mode	Description	Mitigation
Rare-fact loss	Correct but infrequent facts are removed	Rarity score, audit sampling, CCLR tracking
Consensus collapse	Agents imitate the same error	diverse agents, disagreement preservation
Source bias	high-quality-looking source is wrong	provenance, contradiction graph, human audit
Over-compression	nuance is lost in canonicalization	disputed cards and alternative explanations
Cost inversion	refinery cost exceeds savings	amortized accounting and small-scale pilots
Safety leakage	unsafe code becomes training target	safety field and sandbox-only labels

Table 4: Failure modes and mitigations.

19 Implementation Roadmap

A practical roadmap begins with data refining rather than foundation-model training. Phase 1 processes a small Stack Overflow or Linux-documentation subset and measures compression, audit error, and CCLR. Phase 2 trains 50M and 125M models on raw versus CKR data. Phase 3 scales to 500M and isolates attention effects. Phase 4 tests 1B. A 4B run should only happen if smaller stages show consistent gains.

20 Claims and Limits

This proposal does not claim that a 4B model will generally match a frontier-scale model. The expected value is narrower: a domain-specific 4B model trained on high-density refined data may outperform a raw-data model of the same size and may approach a larger generic model in selected domains. The larger claim, that knowledge compression changes scaling behavior, remains unproven.

21 Reviewer Checklist

1. Core scoring and provenance symbols have no conflicting meanings across sections.
2. $\mathcal{R}$ is reserved for refined corpus; $\text{Rar}(g)$ is reserved for rarity.
3. Source quality is $\text{Src}(g)$; evaluation is $\text{Eval}(M, q)$.
4. Evidence is not double counted in vote masses and evidence score.
5. Rejected-card metrics distinguish triage rate (RCF) from actual knowledge-loss rate (CCLR).
6. Attention effect is isolated by A7 vs A8; data effect by A6 vs A7.
7. All references are cited in the body.

8. SubQ is labeled as a vendor technical report, not peer-reviewed evidence.
9. The 4B target is staged, not claimed from 500M results.
10. Equations are rendered in math mode.

22 Conclusion

Consensus Knowledge Refinery proposes that model training should not treat all raw text as equal. It defines a multi-agent, provenance-preserving pipeline for converting noisy corpora into refined knowledge cards. The framework uses signed agreement, source quality, evidence strength, rarity, conflict, and uncertainty to decide which claims should be accepted, rejected, disputed, or audited. The proposal’s strongest claim is not that refinement is automatically superior, but that the hypothesis is testable with controlled ablations. If validated, CKR could reduce duplicated training signal, improve knowledge density, and create a clearer bridge between data quality and model capability.

A Appendix A: Source-Quality Rubric

Source quality is treated as a measurable feature rather than a binary trusted/untrusted label. A source score $Q_s \in [0, 1]$ may be computed from several observable dimensions:

$$Q_s = w_{\text{auth}} \text{Auth}_s + w_{\text{prov}} \text{Prov}_s + w_{\text{rec}} \text{Rec}_s + w_{\text{lic}} \text{Lic}_s + w_{\text{ver}} \text{Ver}_s, \quad (29)$$

where Auth_s is authoritativeness, Prov_s is provenance completeness, Rec_s is recency, Lic_s is license compatibility, and Ver_s is independent verification. Weights should be normalized so $\sum_r w_r = 1$ over rubric dimensions r . For code data, official documentation and tested execution traces receive higher Ver_s than untested forum comments. For scientific data, peer-reviewed sources can receive higher Auth_s but should not automatically override more recent replication evidence.

Dimension	Range	Example interpretation
Auth_s	$[0, 1]$	official docs, peer review, maintainer answers, institutional authority
Prov_s	$[0, 1]$	URL, timestamp, author, version, and license are preserved
Rec_s	$[0, 1]$	source is current for time-sensitive domains
Lic_s	$[0, 1]$	redistribution and training-use compatibility
Ver_s	$[0, 1]$	executable tests, independent agreement, or replication

Table 5: Source-quality rubric. The values are features for scoring, not claims of absolute truth.

B Appendix B: Human Audit Sampling

CKR should not be evaluated only by automatic agreement among agents. Human audit is required because correlated model errors can produce false consensus. A practical audit protocol samples from four buckets: high-confidence accepted cards, high-confidence rejected cards, disputed cards, and uncertain cards. The high-confidence rejected bucket is especially important because it measures potential knowledge loss.

Algorithm 4 Audit sampling protocol

Require: Accepted set $\mathcal{B}_{\text{accept}}$, rejected set $\mathcal{B}_{\text{reject}}$, disputed set $\mathcal{B}_{\text{dispute}}$, uncertain set $\mathcal{B}_{\text{uncertain}}$

- 1: Sample n_A cards uniformly from $\mathcal{B}_{\text{accept}}$
 - 2: Sample n_R cards with probability proportional to $\mathcal{K}(g)$ before rejection
 - 3: Sample n_D disputed cards with high $C(g)$
 - 4: Sample n_U uncertain cards near decision thresholds
 - 5: Human annotators label each sampled card as correct, incorrect, incomplete, or unsafe
 - 6: Update calibration metrics, source-quality weights, and agent reliability
-

The sampling design intentionally over-samples boundary cases and high-risk rejections. A low average error rate can hide dangerous failure modes if rare but important facts are suppressed.

C Appendix C: Worked Example

Consider a coding question: “How do I remove duplicates from a Python list while preserving order?” Raw sources may contain conflicting suggestions: using `set(x)`, using `dict.fromkeys`, or writing a loop. Agents produce votes. The Python specialist supports `dict.fromkeys`. The testing specialist rejects `set(x)` for order-preserving deduplication. The safety specialist marks all options safe-to-run. The `dict.fromkeys` cluster receives high signed agreement and evidence. The `set(x)` cluster may receive a high rejection score because it solves duplicate removal but fails the order-preserving constraint.

The final accepted card should not simply say “use a set.” It should say: *Use `dict.fromkeys` when order must be preserved; use `set(x)` only when order does not matter.* This illustrates the advantage of preserving task constraints during claim extraction.

D Appendix D: Benchmark Tasks

The benchmark suite should include tasks where raw-data duplication is expected to be harmful and tasks where CKR may not help. The goal is not to cherry-pick wins.

Benchmark type	Why included	Expected CKR effect
Factual QA	tests preservation of common and rare facts	moderate gain if duplicates and stale claims are removed
Coding QA	tests best-answer selection and executable correctness	strong gain in domains with many outdated answers
Long-tail facts	tests rare-fact preservation	risk area; CCLR must be monitored
Contradictory claims	tests dispute handling	gain if disputed cards are preserved instead of flattened
Reasoning tasks	tests whether refined data helps reasoning	uncertain; data quality may help but cannot replace capacity
Safety tasks	tests unsafe-code filtering	gain if safety field is used correctly

Table 6: Benchmark task categories and expected effects.

E Appendix E: Ablation Interpretation

Ablations are only useful if each arm changes one factor at a time. A6 versus A7 measures the data effect at 500M parameters because the model size and dense attention are fixed. A7 versus A8 measures the attention effect because data and size are fixed while the attention mechanism changes. A4 measures the role of conflict scoring; if A4 improves average accuracy but worsens CCLR, conflict scoring may be protecting rare correct claims.

Comparison	Isolated factor	Interpretation
A2 vs A3	CKR data at 125M	first meaningful small-model test
A3 vs A4	conflict term	whether contradiction modeling matters
A3 vs A5	rarity term	whether rare-fact protection matters
A6 vs A7	CKR data at 500M	same-size data effect
A7 vs A8	sparse attention	same-data attention effect
A7 vs A9	500M to 1B scaling	cleaner dense-CKR scaling pilot

Table 7: Ablation interpretation guide.

F Appendix F: Cost and Hardware Roadmap

Consumer GPUs should be used to test the refinery and small models, not to train a 4B model from scratch. A staged roadmap reduces risk.

Stage	Model size	Hardware	Goal
S0	none	CPU/RTX 3050	build claim extraction and data cards
S1	50M	RTX 3050 or cheap cloud	smoke-test raw vs CKR
S2	125M	single consumer/cloud GPU	validate metrics and audit pipeline
S3	500M	A100-class short run	isolate data and attention effects
S4	1B	multi-GPU/cloud	test scaling stability
S5	4B	A100/H100-class budget	final validation if earlier stages succeed

Table 8: Hardware roadmap. The 4B stage is conditional, not the starting point.

G Appendix G: Licensing and Provenance

A practical CKR system must treat licensing as a first-class field. If a source cannot be redistributed or used for training, it should not be copied into $\mathcal{R}$. A conservative version of CKR can store only metadata and short derived facts while preserving a pointer to the source. The exact legal status depends on jurisdiction, dataset license, and usage model; this proposal does not provide legal advice.

H Appendix H: Why Consensus Is Not Truth

Consensus is evidence, not truth. A false claim repeated by many low-quality sources can score highly unless source quality, evidence, and audit checks are used. Conversely, a correct rare claim can be rejected if the system over-weights common agreement. CKR therefore uses four safety valves: rarity preservation, dispute cards, human audit, and provenance. The system should prefer “disputed” over “deleted” when evidence is mixed.

I Appendix I: Optimization Objective

The ideal objective is conceptual rather than directly tractable:

$$\mathcal{R}^* = \arg \max_{\mathcal{R} \subseteq \Phi(\mathcal{D})} \mathbb{E}_{q \sim \mathcal{Q}} [\text{Eval}(M_{\mathcal{R}}, q)] - \lambda_c \text{Cost}(\mathcal{R}) - \lambda_l \text{Loss}(\mathcal{R}), \quad (30)$$

where $\Phi(\mathcal{D})$ is the set of candidate refined corpora obtainable from $\mathcal{D}$, $M_{\mathcal{R}}$ is a model trained on $\mathcal{R}$, and $\text{Loss}(\mathcal{R})$ penalizes knowledge loss. This cannot be optimized directly because each candidate $\mathcal{R}$ would require training a model. The practical surrogate is to optimize card-level audit metrics, small-model validation, and staged ablations.

J Appendix J: Open Research Questions

Several questions remain open. Can CKR improve reasoning, or only factual precision? Does concept-level compression remove useful linguistic diversity? How should disputed knowledge be represented during pretraining? Can agent ensembles avoid correlated errors if most agents share similar pretraining data? How much human audit is needed to calibrate source and agent reliability? These questions require empirical work before CKR can be treated as more than a promising hypothesis.

References

- [1] A. Vaswani et al. “Attention Is All You Need.” NeurIPS, 2017.
- [2] T. Dao et al. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.” NeurIPS, 2022.
- [3] S. Ramirez et al. “SubQ-1.1-Small Technical Report.” Subquadratic AI, 2026. Vendor technical report.
- [4] A. Gu and T. Dao. “Mamba: Linear-Time Sequence Modeling with Selective State Spaces.” arXiv:2312.00752, 2023.
- [5] B. Peng et al. “RWKV: Reinventing RNNs for the Transformer Era.” arXiv:2305.13048, 2023.
- [6] H. Wang et al. “BitNet: Scaling 1-bit Transformers for Large Language Models.” arXiv:2310.11453, 2023.
- [7] S. Ma et al. “The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits.” arXiv:2402.17764, 2024.

- [8] S. Ma et al. “BitNet b1.58 2B4T Technical Report.” arXiv:2504.12285, 2025.
- [9] J. Wang et al. “1-bit AI Infra: Part 1.1, Fast and Lossless BitNet b1.58 Inference on CPUs.” arXiv:2410.16144, 2024.
- [10] G. Hinton, O. Vinyals, and J. Dean. “Distilling the Knowledge in a Neural Network.” arXiv:1503.02531, 2015.
- [11] T. Wang et al. “Dataset Distillation.” arXiv:1811.10959, 2018.
- [12] R. Sutton. “The Bitter Lesson.” Essay, 2019.